Assignment no 2

Hadia Azeem

24F-0829

4E

Question no 6

Code :

```
import tkinter as tk
from tkinter import messagebox, simpledialog
import random
import time

class Game:
    def __init__(self, root):
        self.root = root
        self.root.title("Pathfinding")

        # Grid size
        self.rows = 15
        self.cols = 20
        try:
            self.rows = int(simpledialog.askstring("Size", "Rows?") or "15")
            self.cols = int(simpledialog.askstring("Size", "Columns?") or "20")
        except:
            pass

        self.size = 30
```

```
self.grid = []

# Create grid
for i in range(self.rows):
    row = []
    for j in range(self.cols):
        row.append({
            'wall': False,
            'parent': None,
            'visited': False,
            'dist': 999999
        })
    self.grid.append(row)

# Start and end
self.start = (1, 1)
self.goal = (self.rows-2, self.cols-2)
self.player = self.start

# Lists
self.path = []
self.open_list = []
self.closed_list = []

# Flags
self.searching = False
self.moving = False
self.dynamic = False
```

```
    # Counters
    self.visited = 0
    self.cost = 0
    self.search_time = 0

    # GUI
    self.make_buttons()
    self.make_canvas()
    self.draw()

def make_buttons(self):
    top = tk.Frame(self.root)
    top.pack()

    tk.Button(top, text="A*", command=self.astar).pack(side=tk.LEFT, padx=2)
    tk.Button(top, text="Greedy", command=self.greedy).pack(side=tk.LEFT, padx=2)
    tk.Button(top, text="Random", command=self.random_walls).pack(side=tk.LEFT, padx=2)
    tk.Button(top, text="Clear", command=self.clear).pack(side=tk.LEFT, padx=2)
    tk.Button(top, text="Move", command=self.start_move).pack(side=tk.LEFT, padx=2)
    tk.Button(top, text="Stop", command=self.stop_move).pack(side=tk.LEFT, padx=2)

    self.dyn_var = tk.IntVar()
    tk.Checkbutton(top, text="Dynamic", variable=self.dyn_var).pack(side=tk.LEFT, padx=5)

    self.info = tk.Label(top, text="0 | 0 | 0")
    self.info.pack(side=tk.RIGHT, padx=10)

def make_canvas(self):
    w = self.cols * self.size
```

```
    h = self.rows * self.size
    self.canvas = tk.Canvas(self.root, width=w, height=h, bg='white')
    self.canvas.pack()

    self.canvas.bind("<B1-Motion>", self.add_wall)
    self.canvas.bind("<Button-1>", self.add_wall)
    self.canvas.bind("<Button-3>", self.remove_wall)

    tk.Label(self.root, text="Left=Add Wall, Right=Remove").pack()

def draw(self):
    self.canvas.delete("all")
    for i in range(self.rows):
        for j in range(self.cols):
            x1 = j * self.size
            y1 = i * self.size
            x2 = x1 + self.size
            y2 = y1 + self.size

            # Decide color
            if (i, j) == self.player and self.moving:
                color = 'orange'
            elif (i, j) == self.start:
                color = 'green'
            elif (i, j) == self.goal:
                color = 'red'
            elif self.grid[i][j]['wall']:
                color = 'black'
            elif (i, j) in self.path:
```

```
            color = 'yellow'
        elif (i, j) in self.closed_list:
            color = 'light blue'
        elif (i, j) in self.open_list:
            color = 'purple'
        else:
            color = 'white'

        self.canvas.create_rectangle(x1, y1, x2, y2, fill=color, outline='gray')

def add_wall(self, event):
    j = event.x // self.size
    i = event.y // self.size
    if 0 <= i < self.rows and 0 <= j < self.cols:
        if (i, j) != self.start and (i, j) != self.goal:
            self.grid[i][j]['wall'] = True
            self.draw()

def remove_wall(self, event):
    j = event.x // self.size
    i = event.y // self.size
    if 0 <= i < self.rows and 0 <= j < self.cols:
        self.grid[i][j]['wall'] = False
        self.draw()

def get_neighbors(self, pos):
    i, j = pos
    neighbors = []
    for di, dj in [(0,1), (0,-1), (1,0), (-1,0)]:
```

```
        ni, nj = i + di, j + dj
        if 0 <= ni < self.rows and 0 <= nj < self.cols:
            if not self.grid[ni][nj]['wall']:
                neighbors.append((ni, nj))
    return neighbors

def manhattan(self, pos):
    return abs(pos[0] - self.goal[0]) + abs(pos[1] - self.goal[1])

def astar(self):
    self.searching = True
    self.open_list = []
    self.closed_list = []
    self.path = []

    # Reset
    for i in range(self.rows):
        for j in range(self.cols):
            self.grid[i][j]['dist'] = 999999
            self.grid[i][j]['parent'] = None

    self.grid[self.start[0]][self.start[1]]['dist'] = 0
    self.open_list.append(self.start)
    self.visited = 0
    start_time = time.time()

    while self.open_list and self.searching:
        # Find best node
        current = self.open_list[0]
```

```
for pos in self.open_list:
    f1 = self.grid[pos[0]][pos[1]]['dist'] + self.manhattan(pos)
    f2 = self.grid[current[0]][current[1]]['dist'] + self.manhattan(current)
    if f1 < f2:
        current = pos

self.open_list.remove(current)
self.closed_list.append(current)

if current != self.start and current != self.goal:
    self.visited += 1

# Found goal?
if current == self.goal:
    self.search_time = (time.time() - start_time) * 1000
    self.make_path(current)
    self.searching = False
    self.update_info()
    messagebox.showinfo("Done", "Path found!")
    self.draw()
    return

# Check neighbors
for neighbor in self.get_neighbors(current):
    if neighbor in self.closed_list:
        continue

    new_dist = self.grid[current[0]][current[1]]['dist'] + 1
```

```
            if new_dist < self.grid[neighbor[0]][neighbor[1]]['dist']:
                self.grid[neighbor[0]][neighbor[1]]['parent'] = current
                self.grid[neighbor[0]][neighbor[1]]['dist'] = new_dist

                if neighbor not in self.open_list:
                    self.open_list.append(neighbor)

        self.draw()
        self.root.update()
        time.sleep(0.01)

    self.searching = False
    messagebox.showinfo("Sorry", "No path found!")

def greedy(self):
    self.searching = True
    self.open_list = []
    self.closed_list = []
    self.path = []

    # Reset
    for i in range(self.rows):
        for j in range(self.cols):
            self.grid[i][j]['parent'] = None

    self.open_list.append(self.start)
    self.visited = 0
    start_time = time.time()
```

```
while self.open_list and self.searching:
    # Find best node (only heuristic)
    current = self.open_list[0]
    for pos in self.open_list:
        if self.manhattan(pos) < self.manhattan(current):
            current = pos

    self.open_list.remove(current)
    self.closed_list.append(current)

    if current != self.start and current != self.goal:
        self.visited += 1

    # Found goal?
    if current == self.goal:
        self.search_time = (time.time() - start_time) * 1000
        self.make_path(current)
        self.searching = False
        self.update_info()
        messagebox.showinfo("Done", "Path found!")
        self.draw()
        return

    # Check neighbors
    for neighbor in self.get_neighbors(current):
        if neighbor not in self.closed_list and neighbor not in self.open_list:
            self.grid[neighbor[0]][neighbor[1]]['parent'] = current
            self.open_list.append(neighbor)
```

```
        self.draw()
        self.root.update()
        time.sleep(0.01)

    self.searching = False
    messagebox.showinfo("Sorry", "No path found!")

def make_path(self, end):
    self.path = []
    current = end
    self.cost = 0

    while current:
        self.path.append(current)
        if current != self.start and current != self.goal:
            self.cost += 1
        current = self.grid[current[0]][current[1]]['parent']

    self.player = self.start
    self.draw()

def start_move(self):
    if not self.path:
        messagebox.showwarning("Warning", "Find a path first!")
        return
    self.moving = True
    self.player = self.start
    self.move()
```

```
def stop_move(self):
    self.moving = False

def move(self):
    if not self.moving:
        return

    if self.player == self.goal:
        self.moving = False
        messagebox.showinfo("Done", "Goal reached!")
        return

    # Next step
    try:
        idx = self.path.index(self.player)
        if idx < len(self.path) - 1:
            next_pos = self.path[idx + 1]

            # Check wall
            if not self.grid[next_pos[0]][next_pos[1]]['wall']:
                self.player = next_pos
                self.draw()
                self.root.after(300, self.move)
            else:
                if self.dyn_var.get():
                    messagebox.showinfo("Info", "Path blocked! Finding new path...")
                    self.astar()
                    if self.path:
                        self.moving = True
```

```
                self.move()
    except:
        # Player not in path (happens when path changes)
        if self.dyn_var.get():
            self.astar()
            if self.path:
                self.moving = True
                self.move()

def random_walls(self):
    self.clear()
    for i in range(self.rows):
        for j in range(self.cols):
            if random.random() < 0.3:
                if (i, j) != self.start and (i, j) != self.goal:
                    self.grid[i][j]['wall'] = True
    self.draw()

def clear(self):
    self.searching = False
    self.moving = False
    self.path = []
    self.open_list = []
    self.closed_list = []
    self.player = self.start

    for i in range(self.rows):
        for j in range(self.cols):
            self.grid[i][j]['wall'] = False
```

```
            self.grid[i][j]['parent'] = None


        self.visited = 0
        self.cost = 0
        self.search_time = 0
        self.update_info()
        self.draw()


    def update_info(self):
        self.info.config(text=f"{self.visited} | {self.cost} | {self.search_time:.0f}ms")


# Start
window = tk.Tk()
game = Game(window)
window.mainloop()
```

implementation logic:

This project is a grid-based pathfinding simulator made with Python and Tkinter. The player moves from a start position to a goal position while avoiding walls. You can choose different search strategies and change the grid while the player is moving.

Grid Setup

- The grid is a 2D array. Each cell has:
  - wall: if it is blocked
  - parent: to remember the previous cell for the path
  - dist: distance from start used in a*
- Start is at (1,1) and goal is at the bottom right by default.

Search Algorithms

*(A)**

- Combines distance from start (g) and estimated distance to goal (h) with f = g + h
- Uses open list and closed list

- Picks the node with the lowest f value and explores neighbors
- Finds the shortest path if the heuristic is correct
- Can recalculate path if a wall blocks it during movement

Greedy Best-First Search

- Uses only the estimated distance to goal (h)
- Picks the node that looks closest to the goal
- Faster in open grids but may not find the shortest path
- Can fail in complex grids

Dynamic Movement

- Player moves along the path automatically
- Dynamic mode recalculates the path if a wall blocks it

Pros and Cons

*(A)**
Pros:

- Finds shortest path
- Works even if walls appear

Cons:

- Slower than Greedy
- Uses more memory

Greedy Best-First Search
Pros:

- Fast in open spaces
- Easy to use

Cons:

- Path may not be shortest
- Can get stuck in complex grids

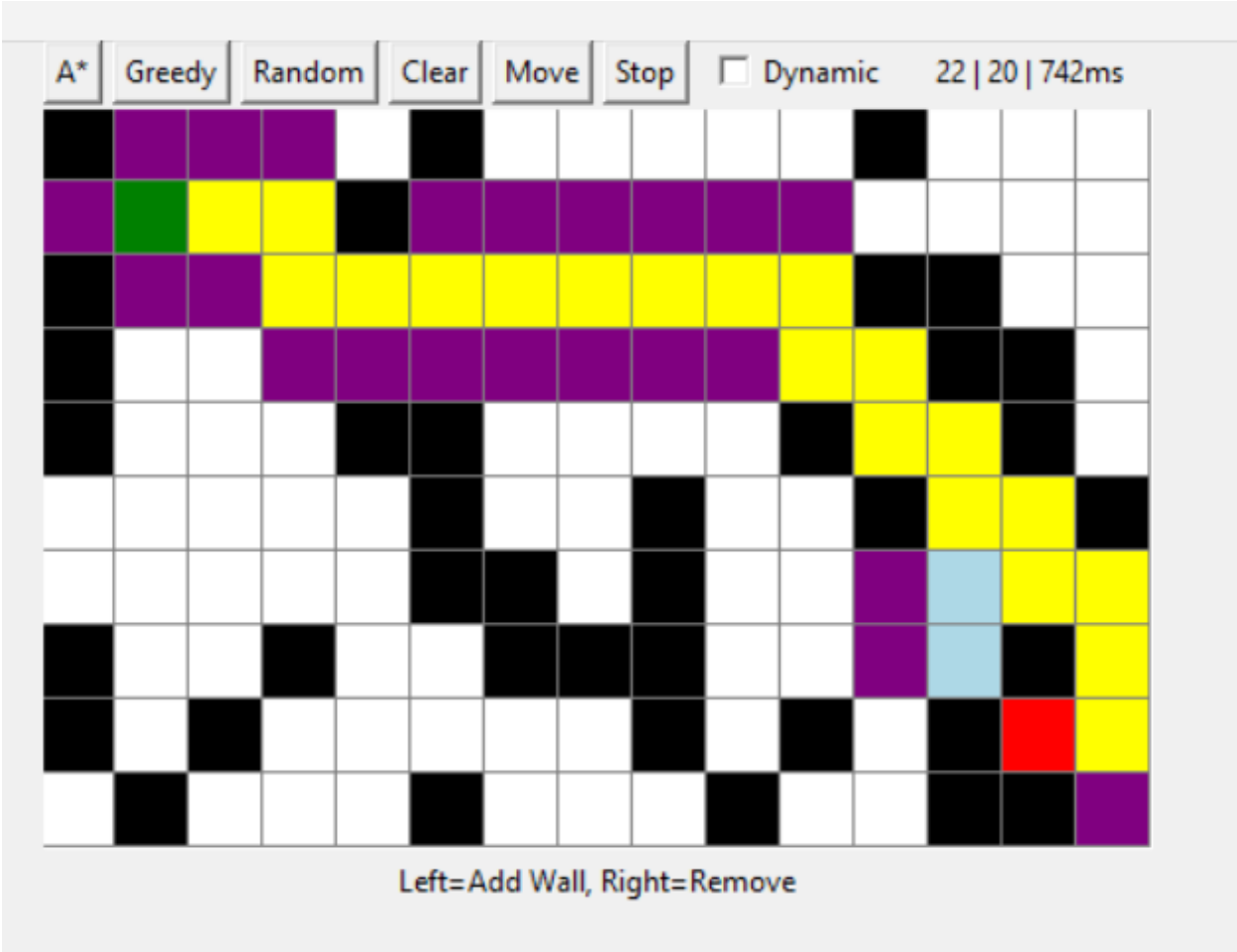
Dynamic Mode
Pros:

- Handles walls that appear while moving

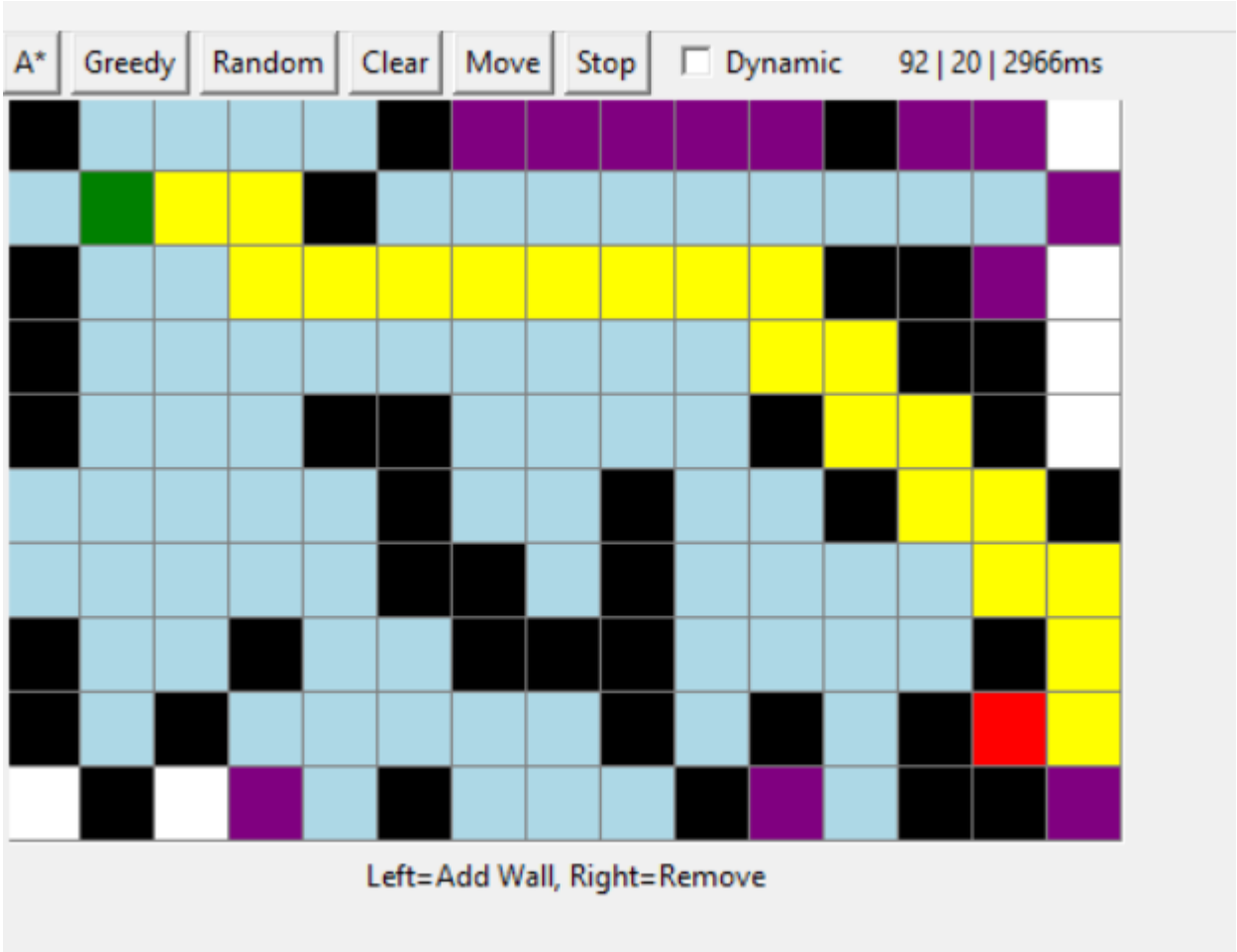
- Player keeps moving

Cons:

- Recalculating path can be slow in large grids



=


A*
Greedy
Random
Clear
Move
Stop
Dynamic
92 | 20 | 2966ms
Left=Add Wall, Right=Remove